

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I [Priyanka.G / 192524124](#) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

Student 31: Priyanka G | Register Number: 192524124

1. Scenario

A disaster-response agency deploys AI-guided drones to airlift medical supplies to villages cut off by a wildfire. The environment is highly dynamic - access roads and forest trails may become impassable without warning, and smoke conditions affect travel cost. The drone has access to: a heuristic estimate (straight-line distance to each village), partial real-time updates about newly blocked trails, and variable movement costs depending on terrain and visibility. Due to the urgency of the situation, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search (GBFS) in Dynamic Environments

Mechanism & Evaluation Function:

- Greedy Best-First Search expands the node that appears closest to the goal based solely on a heuristic function: $f(n) = h(n)$
where $h(n)$ represents the straight-line distance from node n to the target village. GBFS completely ignores the path cost accumulated so far ($g(n) = 0$).

Strengths

- **Computational Speed:** In benign, unobstructed environments, GBFS reaches the goal quickly by greedily following the heuristic gradient toward the target.
- **Low Memory Footprint:** Keeps the open list focused on nodes along the promising path, requiring fewer node expansions than uninformed methods.

Weaknesses Under Uncertainty

- **Lack of Optimality:** Because $g(n)$ is ignored, GBFS may choose a direct straight line into a severe smoke zone or rough terrain, resulting in extreme flight delays or battery exhaustion.
- **Incompleteness & Trapping:** If a trail is suddenly blocked by wildfire, GBFS can get trapped in local loops or dead-ends, continuously trying to move closer spatially while ignoring the fact that it is physically blocked.
- **Inability to Backtrack Efficiently:** GBFS does not re-evaluate choices when actual movement costs spike dynamically.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.

A* Search Handling of Cost and Heuristic Balance

Mechanism & Evaluation Function

A* Search evaluates nodes using a combined cost function:

- $f(n) = g(n) + h(n)$
- $g(n)$: Exact accumulated cost from the start node to node n (reflecting actual flight time, wind resistance, terrain difficulty, and smoke delays).
- $h(n)$: Admissible and consistent estimated cost from node n to the destination village (straight-line distance).

[Start]

/ \

$g=10, h=50$ $g=30, h=20$

/ \

$f(A) = 60$ $f(B) = 50$ <-- A* expands B first

Balancing Cost and Heuristic

- **Backward Cost ($g(n)$):** Guarantees that the algorithm penalizes paths that pass through high-risk, high-cost wildfire areas.
- **Forward Cost ($h(n)$):** Directs the search toward the destination, preventing unnecessary exploration in directions away from the target.
- **Behavior in Dynamic Wildfire Conditions**
- If smoke elevates the travel cost of a route ($g(n)$ increases significantly), the total $f(n)$ value rises. A* automatically redirects the drone to alternate paths whose $f(n)$ values are lower, ensuring optimal and safe routing.

iii. Analyze the impact of heuristic accuracy on both algorithms.

Impact of Heuristic Accuracy on GBFS and A*

Heuristic Condition	Impact on A* Search	Impact on Greedy Best-First Search
Perfect Heuristic ($h(n) = h^*(n)$)	Expands only the nodes on the optimal path. Time complexity drops from exponential to $O(d)$.	Directs search straight to the goal along the optimal path with zero redundant exploration.
Underestimating / Admissible ($h(n) \leq h^*(n)$)	Guarantees finding the optimal (shortest/safest) path. Lower $h(n)$ increases node expansions toward Dijkstra's algorithm ($h=0$).	Still prone to sub-optimal detours because it ignores $g(n)$, regardless of admissibility.
Overestimating / Inadmissible ($h(n) > h^*(n)$)	Loses optimality guarantee. May return a sub-optimal route prematurely to save computation time.	Behaves erratically; overestimations misguide the search into local traps or high-cost zones.

iv. Discuss how dynamic changes (newly blocked trails) affect search performance.

Impact of Dynamic Changes (Newly Blocked Trails)

- **Path Invalidation:** When a forest trail is blocked mid-flight, the edge weight on the search graph effectively becomes $c(u, v) = \text{infinity}$.
- **High Re-planning Overhead (Standard Offline A*):** Traditional A* assumes a static graph. If a trail closes, standard A* must throw away its search tree and restart from scratch from the current drone position, causing latency and drone hovering delays.
- **Thrashing in GBFS:** GBFS may oscillate repeatedly near the obstacle because its $h(n)$ values continuously pull it back toward the closed trail.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Recommendation

D* Lite (Dynamic A* Lite) or LSS-LRTA* (Local Search Space Learning Real-Time A*).

Detailed Justification

1. **Incremental Replanning:** Unlike static A*, D* Lite maintains g-values and rhs-values from previous search iterations. When a trail is blocked by wildfire, it incrementally updates only the affected nodes in the local graph rather than recomputing the entire search tree.
2. **Real-Time Safety & Low Latency:** Recomputation takes milliseconds, allowing the drone to make immediate mid-flight adjustments without pausing or hovering in high-risk zones.
3. **Optimality and Bounded Cost:** Guarantees an optimal or bounded sub-optimal detour relative to the updated map state, balancing speed, safety, and energy efficiency.

2. Scenario

A large agricultural enterprise implements an AI system to optimize water distribution through a network of smart irrigation valves across thousands of acres. The objective is to minimize water wastage, energy consumption, and crop stress. The system must continuously adjust valve settings based on soil-moisture sensor readings. However, the search space is extremely large, weather patterns change dynamically, and the system may get stuck in locally optimal valve configurations.

Tasks:

Formulate this as an optimization problem and explain why traditional search is inefficient.

Optimization Problem Formulation

- **State Space Definition**

Let the state X be represented as a high-dimensional vector of continuous valve opening percentages:

$X = [v_1, v_2, v_3, \dots, v_n]^T$, where $v_i \in [0, 100\%]$, for all $i \in \{1, 2, 3, \dots, n\}$

- **Objective / Fitness Function**

Minimize total systemic cost $F(X)$:

$F(X) = w_1 \text{CropStress}(X) + w_2 \text{WaterWastage}(X) + w_3 * \text{EnergyCost}(X)$

where w_1, w_2, w_3 are penalty weights adjusted according to weather conditions.

- **Why Traditional Tree/Graph Search Fails**

- **Continuous and Infinite Search Space:** Traditional search algorithms (A^* , BFS, DFS) require discrete graph nodes and explicit goal states. Valve states are continuous values in $\mathbb{R}^n$, resulting in an infinite branching factor.
- **Combinatorial Explosion:** Discretizing $n = 1000$ valves into 10 step intervals yields 10^{1000} possible states far exceeding the total subatomic particles in the observable universe.

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing Working Mechanism & Limitations

- **Working Mechanism**

- Initialize valve settings X_0 based on current baseline schedule.
- Evaluate current cost $F(X_0)$.
- Generate neighboring configurations X' by applying small random delta changes $\pm \Delta v$ to valve settings.
- Compute $F(X')$. If $F(X') < F(X_0)$ (improvement), set $X_0 \leftarrow X'$.
- Repeat until no immediate neighbor yields a lower cost.

- **Core Limitations**

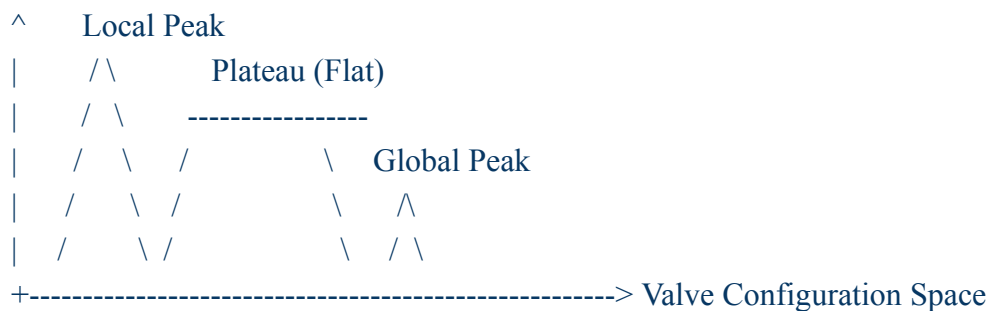
- **Greedy / Myopic:** Accepts only immediate strictly improving moves; has no memory or lookahead.
- **Inability to Escape Inefficient Configurations:** Easily trapped by local structures in the objective landscape.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

Real-World Interpretation of Optimization Pitfalls

Objective

Utility



1. Local Maxima / Minima

- **Real-World Meaning:** A specific valve configuration optimizes moisture for one localized crop zone, but starves a downstream zone due to pipe pressure drops.
- **Impact:** Hill Climbing stops here because adjusting any single valve locally increases overall system cost, missing the globally optimal valve distribution across the entire farm.

2. Plateaus

- **Real-World Meaning:** Large sub-regions where changing valve flow rates by small percentages ($\pm 2\%$) yields zero measurable change in soil sensors due to absorption delays.
- **Impact:** The local gradient becomes zero ($\nabla F(X) = 0$). Hill Climbing degrades into a random, aimless walk and terminates prematurely.

3. Ridges

- **Real-World Meaning:** Correlated dependencies between parameters—such as needing to increase Valve A while simultaneously decreasing Valve B to maintain pipeline pressure stability.
- **Impact:** Hill Climbing adjusts variables one by one or orthogonally, causing it to bounce back and forth across the ridge without making forward progress along the crest.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Advanced Algorithms: Overcoming Local Search Traps:

Simulated Annealing (SA)

Simulated Annealing prevents trapping by allowing non-improving (worse) moves controlled by a temperature parameter T : $P(\text{accept } \Delta E) = \exp(-\Delta E / T)$

- **Early Phase (High T):** The algorithm accepts worse configurations frequently, exploring the space and jumping out of local minima and plateaus.
- **Cooling Phase ($T \rightarrow 0$):** The probability of accepting worse moves drops, causing SA to converge precisely onto a high-quality global optimum.

Genetic Algorithms (GA)

GA uses a population-based approach:

- **Chromosome:** A vector representing a full valve array configuration.
- **Selection:** Fitter configurations survive based on lower cost $F(X)$.
- **Crossover:** Combines effective valve configurations from distinct fields (e.g., merging Zone A's optimal schedule with Zone B's schedule).
- **Mutation:** Introduces random valve adjustments, maintaining genetic diversity and breaking out of plateaus and ridges.

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

Recommended Approach

Genetic Algorithm combined with Local Search (Memetic Algorithm) OR Parallel Simulated Annealing.

Justification

- **High Dimensionality Handling:** Genetic Algorithms naturally maintain a diverse population of solutions across high-dimensional continuous search spaces (1000+ valves).
- **Adaptability to Dynamic Weather:** When weather predictions shift, the existing population provides a rich set of pre-evaluated candidates, enabling rapid re-adaptation compared to single-point searches.
- **Scalability:** GA processes subpopulations independently across distributed controllers (edge computing at irrigation hubs), bypassing single-thread bottlenecks.

3. Scenario

An autonomous submarine is deployed to explore an uncharted ocean trench. It must: navigate safely to map the trench floor, avoid hazards (rock formations, thermal vents), operate with limited sonar range, and make decisions without a complete map. The submarine updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search.

Online Search vs. Offline Search

- **Offline Search:** [Complete Map] -> [Compute Full Path] -> [Execute Actions]
- **Online Search:** [Sense Local] -> [Compute Step] -> [Move & Map]
- **Offline Search Failure:** Offline algorithms (A*, Dijkstra) compute a complete path from start to goal before moving. This requires a complete, static environment map.
- **Necessity of Online Search:** In an uncharted trench, the map does not exist prior to exploration. The submarine must interleave sensing, local computation, and action execution in real-time, updating its path step-by-step as sonar discovers new terrain features.

- ii. Analyze the challenges of partial observability and dynamic environments.

Challenges of Partial Observability & Dynamic Environments

- **Partial Observability:** The submarine's sensors only perceive surroundings within sonar range R. Unobserved areas represent high structural uncertainty. The agent must operate on a Belief State rather than an exact world state.
- **Dynamic Environment Hazards:** Thermal vent plumes and rockslides alter passability dynamically. A path recorded as clear 10 minutes ago may become hazardous.
- **Perceptual Aliasing:** Identical rock walls across different parts of the trench can cause localization drift unless managed continuously.

- iii. Describe how the submarine builds and updates its internal model.

Internal Model Construction and State Updating

Spatial Representation (Occupancy Grid Mapping)

The submarine discretizes the 3D space into a spatial volume grid where each cell c_i stores an occupancy probability: $P(c_i = \text{Occupied})$ in $[0, 1]$

Sensor Integration via SLAM

Uses Simultaneous Localization and Mapping (SLAM) (e.g., Extended Kalman Filter or Particle Filter SLAM):

- **Predict Phase:** Estimates new submarine position (x, y, z, θ) using Inertial Measurement Unit (IMU) dead reckoning.
- **Update Phase:** Corrects position and updates cell occupancy values using incoming sonar range data.
- **Bayesian Belief Updates**
- When new sonar reading z_t arrives, the occupancy probability of cell m_i is updated recursively: $P(m_i | z_{1:t}) = [P(z_t | m_i) * P(m_i | z_{1:t-1})] / P(z_t | z_{1:t-1})$

iv. Compare online search with uninformed and informed search strategies.

Comparison of Search Strategies

Criteria	Uninformed Search (e.g., BFS / DFS)	Informed Search (e.g., A*)	Online Search (e.g., LRTA*)
Map Requirement	Requires complete prior map	Requires complete prior map	Works with zero initial map
Execution Model	Plan-then-act	Plan-then-act	Interleaves planning and acting
Handling Obstacles	Fails on unexpected barriers	Fails on unexpected barriers	Dynamically re-plans around hazards
Memory Footprint	Exponential in depth	Stores all expanded nodes	Bounded memory per step
Suitability for Submarine	Unsuitable (No prior map)	Unsuitable (No prior map)	Ideal (Explores unknown space safely)

v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Recommended Approach

LRTA* (Learning Real-Time A*) integrated with a SLAM-backed Occupancy Grid Map.

Detailed Justification

- Completeness in Unknown Environments:** LRTA* guarantees reaching the destination in finite state spaces by updating heuristic values $h(s)$ of visited states:
$$h(s) \leftarrow \max(h(s), \min_{a \in A(s)} [c(s, a) + h(s')])$$

This prevents the submarine from getting stuck in infinite exploration loops or dead-end trench pockets.
- Real-Time Bounded Decision Time:** LRTA* limits search depth per step, making action decisions within strict time bounds—a critical requirement to prevent collisions with moving hazards or thermal vents.
- Safety and Adaptability:** Seamlessly updates path plans as SLAM populates previously unobserved cells into the occupancy grid, balancing exploration of unknown areas with exploitation of mapped safe routes.